

Every Student Gets the Same Classroom

github.com/seancnc003/UNLVdockerIDES
hub.docker.com/r/seancnc

Does the same programming environment run on every student’s laptop? Browser-based Docker IDEs for UNLV’s C++ and x86-64 assembly courses, tested on the operating systems and processors students own

Sean Cuenco · Faculty Mentor: James Andro-Vasko, Ph.D. · Department of Computer Science, University of Nevada, Las Vegas · Office of Undergraduate Research SURF 2026

The problem: week one is lost to setup

Every systems course starts the same way: a compiler, an assembler, a debugger, and an editor have to work on dozens of different laptops before anyone learns anything. Prior studies measured 75–100 minutes of class time per semester lost to installs [6], support traffic dominated by “it works on my machine” [3], and students with Apple Silicon Macs unable to run the course’s virtual machine at all [4]. At UNLV this touches several hundred CS 135 and CS 218 students every year.

UNLV followed the same path as everyone else, one step behind

- ### 1. Shared login servers (SSH, PuTTY)

bobby · sally · cardiac. Now decommissioned, and the wiki that documented them was deleted in 2025.
- ### 2. Virtual machines, one per course

CS 218 today: a 19 GB VirtualBox image for Intel and an experimental 2023 image for Apple Silicon so slow the course warns the editor “may not be usable.” Official advice for M1/M2 owners: go to the lab with a USB drive.
- ### 3. A container on the student’s own laptop (this project)

One 552 MB download, one command, VS Code in a browser tab. Files stay in a normal folder on the laptop. No server, no account, no VPN, works offline.

Every approach hands the setup burden to someone

Approach	Who carries it	What the student pays
Shared servers (UNLV, CSN)	The department	Needs network + VPN; everything vanishes when the server is retired
Course virtual machines (UNLV, Harvard 2011 [2], Reading [5])	The student’s hardware	Multi-GB images, slow boots, disk corruption; broken on Apple Silicon
Cloud editor (Harvard CS50 on GitHub Codespaces [1])	A vendor, a grant, an account	Internet required; students leave “not sure how to set up an IDE on my computer”
Monitored platform (CodeDive, South Korea [8])	The university’s cluster, and the student’s privacy	Every command and every 1-second typing pause is recorded
Local container (this project; cf. UCSD [3], Florida Tech [4])	The student, once: install Docker	One download, one command, then nothing

What we built

Two Docker images, one per course, each packing VS Code (served to the browser) with the exact tools the course expects: g++ for CS 135; yasm, nasm, ld, and gdb for CS 218. The instructor pins every version. The student installs Docker Desktop once, runs one command, and opens a browser tab. Every AI feature in the editor is switched off, per course policy.

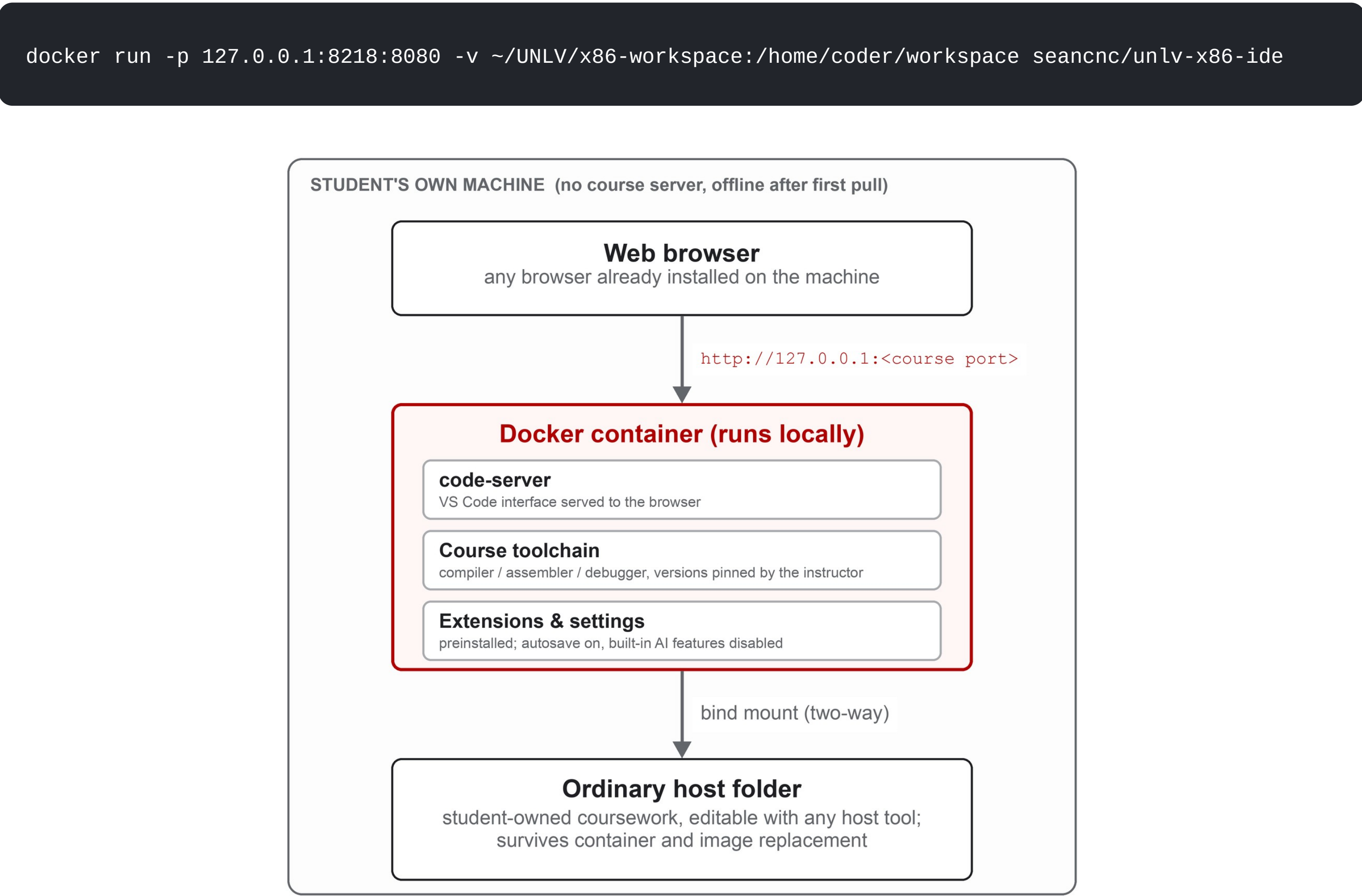


Figure 1. Everything runs on the student’s machine. Coursework lives in an ordinary folder that survives deleting or updating the container.

How we tested it: same exam, four kinds of computer

We cannot borrow one of every laptop students own, so we rented them: for a few dollars total, AWS and Azure supplied a fresh Intel Linux machine, an ARM Linux machine, and a Windows 11 machine, and a base-model M1 MacBook Pro stood in for Apple Silicon. On each, one published script performed a student’s first day (download, open the editor, write, assemble, run, debug) and then built and ran four real CS 218 assignments, writing down everything it observed.

Windows 11 PC	Linux PC	Apple Silicon Mac	Linux on ARM
Intel / AMD	Intel / AMD	M1 Pro, 16 GB	AWS Graviton
✓ all assignments	✓ all assignments	✓ all assignments	✓ all assignments†
✓ debugger	✓ debugger	✗ debugger silent*	✗ debugger silent*

* Under translation the debugger appears to finish but writes no values, so debugger assignments need an Intel/AMD machine; the student handout says so.
† Needs Docker’s own translator; the one that ships with Ubuntu 24.04 crashed, which we reproduced and documented.

What we found

4 of 4

kinds of computer ran every real CS 218 assignment correctly, including the multithreaded one

Identical

the same 552 MB file, byte for byte, on Windows, macOS, and Linux: the promise a VM could never make

0.5 s

until the editor is ready on Intel/AMD machines; about 5 s on Apple Silicon; a VM boots in minutes

552 MB

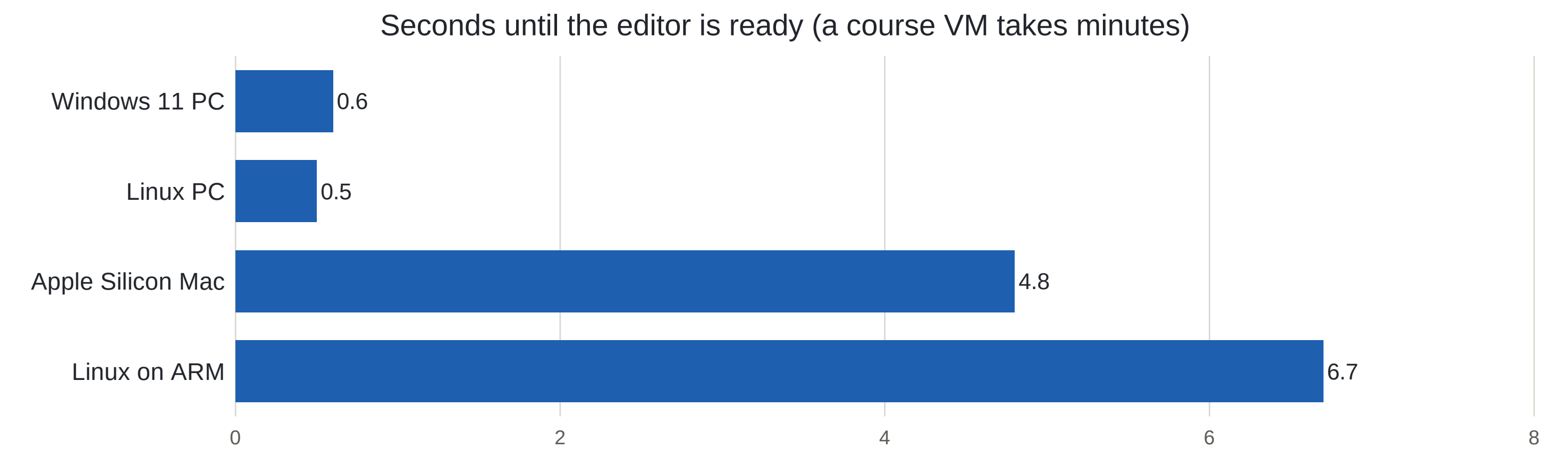
one-time download, versus the 19 GB virtual machine the course distributes today

< \$10

to rent every test machine, failed attempts included; students pay nothing and need no account

1 limit

the debugger goes silent on Apple Silicon and ARM, and we can say exactly why



Why the limit exists, in one sentence. The assembly image is x86-64 on purpose, because the course teaches x86-64; on ARM chips it runs through a translator, and today’s translators run the programs correctly but cannot support the debugger. Windows and Intel machines, most of the class, get everything. Under translation a debugger script still “succeeds” and writes its output file, but with none of the values, which is why the student handout tells Apple Silicon owners to produce debugger deliverables on an Intel machine.

What comes next

- Run it in a real section of CS 218 and CS 135 and measure what only a classroom can: how many students reach a working setup without help, how many staff minutes it saves, and how students feel about it, using survey instruments adapted from [3, 4, 6]. A three-student pilot class has already run on it.
- Build a second assembly image for ARM chips that keeps the editor native and translates only the student’s program, which could bring the debugger back to Apple Silicon and Windows-on-ARM laptops.
- Reuse the rent-four-computers, one-script method to evaluate environments for other courses (Operating Systems next).

References

[1] Malan. Containerizing CS50. ITICSE 2024.
[2] Malan. From Cluster to Cloud to Appliance. ITICSE 2013.
[3] Valstar, Griswold, Porter. Using DevContainers to Standardize Student Development Environments. ITICSE 2020.
[4] Fernalld, O’Connor, Sudhakaran, Nur. Lightweight Symphony. SIGITE 2023.
[5] Cadenas et al. Virtualization for Cost-Effective Teaching of Assembly Language Programming. IEEE Trans. Educ. 2015.
[6] Harvie, Cody, Morrell, Estes. Using Virtual Machines to Enhance the Educational Experience. SIGITE 2019.
[8] Park et al. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring. Applied Sciences 2025.